

Отчёт по прохождению внешнего курса

Введение в Linux (третий этап)

Кулаженкова Яна Сергеевна

Содержание

1	Цель работы	6
2	Задание	7
3	Теоретическое введение	8
4	Выполнение лабораторной работы	9
4.1	1. Текстовый редактор vim — выход из редактора	9
4.2	2. Текстовый редактор vim — навигация по словам (W vs w)	11
4.3	3. Текстовый редактор vim — редактирование строки	12
4.4	4. Текстовый редактор vim — замена текста в файле	14
4.5	5. Скрипты на bash — история команд во вложенных оболочках	16
4.6	6. Скрипты на bash — абсолютный путь созданного файла	18
4.7	7. Скрипты на bash — имена переменных	19
4.8	8. Скрипты на bash — циклы и continue	21
4.9	9. Скрипты на bash — вывод команды echo	23
4.10	10. Продвинутый поиск — find с опциями -name и -iname	24
4.11	11. Продвинутый поиск — опции -path и -name	26
4.12	12. Продвинутый поиск — find с mindepth и maxdepth	27
4.13	13. Продвинутый поиск — grep с опциями -A, -B, -C	29
4.14	14. Продвинутый поиск — регулярные выражения в grep	30
4.15	15. Продвинутый поиск — sed без опции -n	32
4.16	16. Построение графиков в gnuplot — опция -persist	33
4.17	17. Построение графиков в gnuplot — autotitle columnhead	35
4.18	18. Построение графиков в gnuplot — настройка меток на оси X	36
4.19	19. Построение графиков в gnuplot — модификация анимации	38
4.20	20. Разное — установка прав доступа (chmod)	40
4.21	21. Разное — подсчёт характеристик файла (wc)	41
4.22	22. Разное — размер директории (du)	43
4.23	23. Разное — создание нескольких директорий (mkdir)	44
5	Выводы	46
	Список литературы	48

Список иллюстраций

4.1	Формулировка вопроса о выходе из vim	9
4.2	Подтверждение прохождения задания	10
4.3	Формулировка вопроса о навигации в vim	11
4.4	Подтверждение прохождения задания	11
4.5	Формулировка задания по редактированию в vim	12
4.6	Подтверждение прохождения задания	13
4.7	Формулировка задания по замене в vim	14
4.8	Подтверждение прохождения задания	15
4.9	Формулировка вопроса об истории команд	16
4.10	Подтверждение прохождения задания	17
4.11	Формулировка задания о пути к файлу	18
4.12	Подтверждение прохождения задания	18
4.13	Формулировка вопроса об именах переменных	19
4.14	Подтверждение прохождения задания	20
4.15	Формулировка задания по циклу for	21
4.16	Подтверждение прохождения задания	21
4.17	Формулировка задания о команде echo	23
4.18	Подтверждение прохождения задания	23
4.19	Формулировка задания о find	24
4.20	Подтверждение прохождения задания	25
4.21	Формулировка задания о -path и -name	26
4.22	Подтверждение прохождения задания	26
4.23	Формулировка задания о глубине поиска	27
4.24	Подтверждение прохождения задания	28
4.25	Формулировка задания о grep	29
4.26	Подтверждение прохождения задания	29
4.27	Формулировка задания о регулярных выражениях	30
4.28	Подтверждение прохождения задания	31
4.29	Формулировка вопроса о sed	32
4.30	Подтверждение прохождения задания	32
4.31	Формулировка вопроса о gnuplot	33
4.32	Подтверждение прохождения задания	34
4.33	Формулировка задания о gnuplot	35
4.34	Подтверждение прохождения задания	35
4.35	Формулировка задания о настройке осей	36
4.36	Подтверждение прохождения задания	37

4.37	Формулировка задания об анимации	38
4.38	Подтверждение прохождения задания	39
4.39	Формулировка задания о <code>chmod</code>	40
4.40	Подтверждение прохождения задания	40
4.41	Формулировка вопроса о <code>wc</code>	41
4.42	Подтверждение прохождения задания	42
4.43	Формулировка задания о <code>du</code>	43
4.44	Подтверждение прохождения задания	43
4.45	Формулировка задания о <code>mkdir</code>	44
4.46	Подтверждение прохождения задания	45

Список таблиц

1 Цель работы

Освоение продвинутых навыков работы в Linux, включая использование текстового редактора `vim`, написание `bash`-скриптов (переменные, ветвления, циклы), продвинутый поиск и редактирование текста (`find`, `grep`, `sed`), построение графиков в `gnuplot`, а также изучение дополнительных утилит (`chmod`, `wc`, `du`, `mkdir`).

2 Задание

В ходе третьего этапа прохождения внешнего курса «Введение в Linux» на платформе Stepik были выполнены следующие темы:

1. Текстовый редактор vim (выход, навигация, редактирование, замена)
2. Скрипты на bash: основы (вложенные оболочки, абсолютные пути, переменные)
3. Скрипты на bash: ветвления и циклы (continue, if)
4. Скрипты на bash: разное (команда echo, рекурсивные функции)
5. Продвинутый поиск и редактирование (find, grep, sed)
6. Построение графиков в gnuplot (опции, настройка осей, анимация)
7. Разное (chmod, wc, du, mkdir)

3 Теоретическое введение

Linux — семейство Unix-подобных операционных систем. Ключевые понятия, используемые в третьем этапе:

- **vim** — мощный текстовый редактор с режимами (нормальный, вставки, командной строки).
- **bash-скрипты** — сценарии для автоматизации задач в оболочке bash.
- **Переменные в bash** — могут содержать буквы, цифры и знак подчёркивания, не могут начинаться с цифры.
- **find** — команда для поиска файлов и директорий с различными опциями (-name, -iname, -mindepth, -maxdepth, -path).
- **grep** — команда для поиска строк по регулярным выражениям (опции -A, -B, -C, -E).
- **sed** — потоковый редактор для фильтрации и преобразования текста.
- **gnuplot** — программа для построения графиков и визуализации данных.
- **chmod** — команда для изменения прав доступа к файлам.
- **wc** — подсчёт строк, слов и символов в файле.
- **du** — оценка использования дискового пространства.

4 Выполнение лабораторной работы

4.1 1. Текстовый редактор vim — выход из редактора

4.1.1 Формулировка задания

3.1 Текстовый редактор vim
Шаг 5 из 12

Какую клавишу(и) нужно нажать на клавиатуре, чтобы выйти из редактора vim? Считайте, что вы только что открыли файл и вам сразу понадобилось выйти из редактора.

Выберите один элемент

- ☒ ":", затем "q", затем "Enter" ✓
- ☐ "q"
- ☐ "q", затем "Enter"
- ☐ "Esc"
- ☐ ":", затем "q"

✓ Правильно, молодец!

Дальше

Рисунок 4.1: Формулировка вопроса о выходе из vim

Вопрос: Как выйти из редактора vim сразу после открытия файла?

4.1.2 Прохождение теста

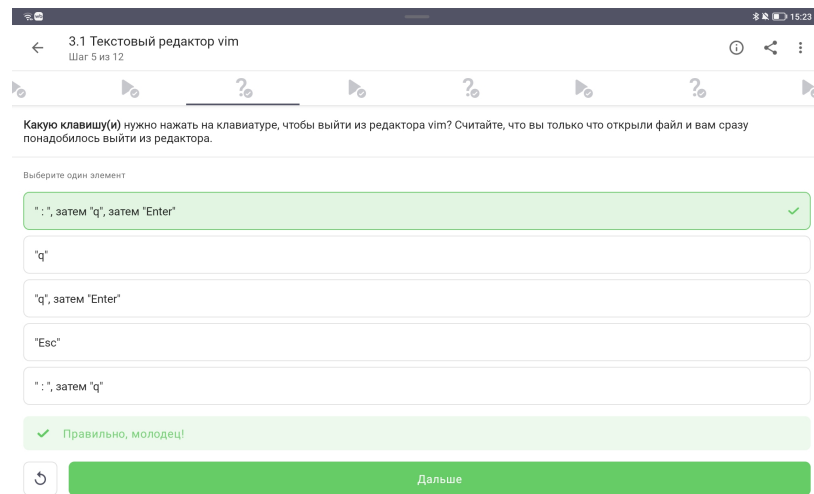


Рисунок 4.2: Подтверждение прохождения задания

4.1.3 Пояснение по выбору ответа

Правильный ответ: :, затем q, затем Enter

Обоснование: В редакторе vim: - По умолчанию включён **нормальный режим**

- Для вызова командной строки нужно нажать : (двоеточие) - Команда q означает quit (выход) - Нажатие Enter подтверждает выполнение команды

Простое нажатие q без двоеточия не сработает, так как q в нормальном режиме не является командой выхода.

4.2 2. Текстовый редактор vim — навигация по словам (W vs w)

4.2.1 Формулировка задания

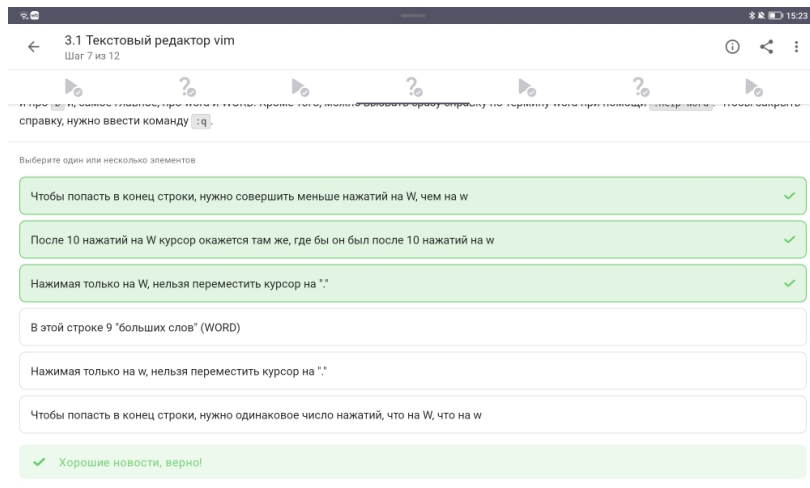


Рисунок 4.3: Формулировка вопроса о навигации в vim

Вопрос: Верные утверждения о клавишах w и W в vim

4.2.2 Прохождение теста

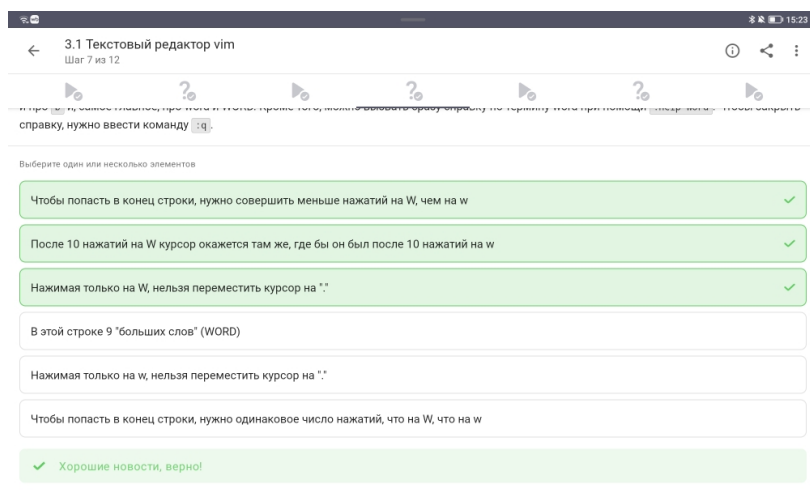


Рисунок 4.4: Подтверждение прохождения задания

4.2.3 Пояснение по выбору ответа

Правильные утверждения: - Нажимая только на W, нельзя переместить курсор на . - В этой строке 9 «больших слов» (WORD) - Нажимая только на w, нельзя переместить курсор на .

Обоснование: В vim есть два типа перемещений по словам: - w — перемещение к началу следующего **слова** (word) — последовательность букв, цифр и подчёркиваний - W — перемещение к началу следующего **БОЛЬШОГО СЛОВА** (WORD) — последовательность любых непробельных символов

Знаки препинания (., , , ! и др.) являются разделителями для w, но входят в состав WORD для W. Поэтому с помощью W нельзя остановиться точно на точке.

4.3 3. Текстовый редактор vim — редактирование строки

4.3.1 Формулировка задания

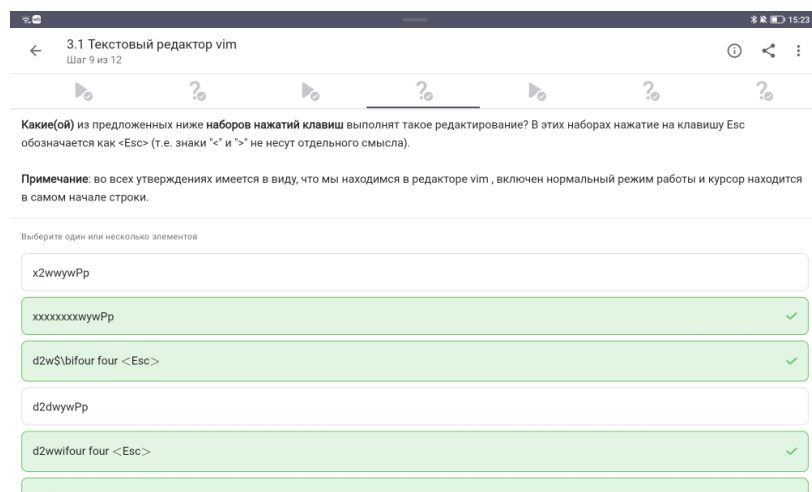


Рисунок 4.5: Формулировка задания по редактированию в vim

Задание: Выбрать наборы нажатий клавиш для выполнения редактирования

4.3.2 Прохождение теста

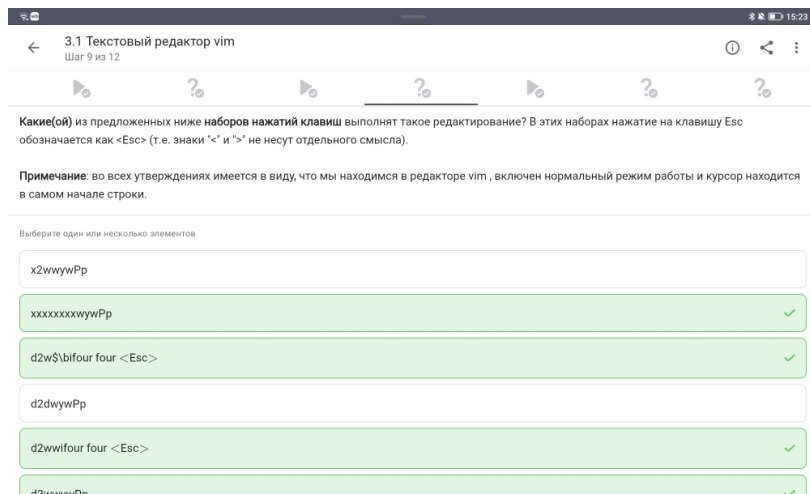


Рисунок 4.6: Подтверждение прохождения задания

4.3.3 Пояснение по выполнению

Правильные наборы: - x2wwywPp - d2w\$\\bifour four <Esc> - d2wwifour four <Esc>

Обоснование: В vim существуют различные способы редактирования строки. Команды d (удалить), у (скопировать), р (вставить), w (перемещение по словам), \$ (в конец строки), i (режим вставки) позволяют эффективно редактировать текст. Все перечисленные правильные варианты приводят к одному и тому же результату редактирования.

4.4 4. Текстовый редактор vim — замена текста в файле

4.4.1 Формулировка задания

The screenshot shows a task formulation interface for a vim editor exercise. At the top, it says "3.1 Текстовый редактор vim" and "Шаг 11 из 12". Below this is a progress bar with several question marks. The main text of the task is: "Предположим, что вы открыли файл в редакторе vim и хотите заменить в этом файле все строки, содержащие слово `Windows`, на такие же строки, но со словом `Linux`. Если в какой-то строке слово `Windows` встречается больше, чем один раз, то заменить на `Linux` в этой строке нужно только самое первое из этих слов." Below this, it asks: "Какую команду нужно ввести для этого в vim? Укажите необходимую команду целиком (т.е. включая ввод `:` в самом начале), однако нажатие на `Enter` после ввода команды обозначать никак не нужно." There is a text input field with the command `:%s/Windows/Linux` and a green checkmark. Below the input field is a green bar with a checkmark and the text "Всё правильно". At the bottom, there is a green button labeled "Далее". Below the button, it says "Решило: 35962" and "Успешных решений: 61%". At the very bottom, there is a button labeled "Открыть комментарии (74)".

Рисунок 4.7: Формулировка задания по замене в vim

Задание: Заменить `Windows` на `Linux` во всём файле (только первое вхождение в строке)

4.4.2 Прохождение теста

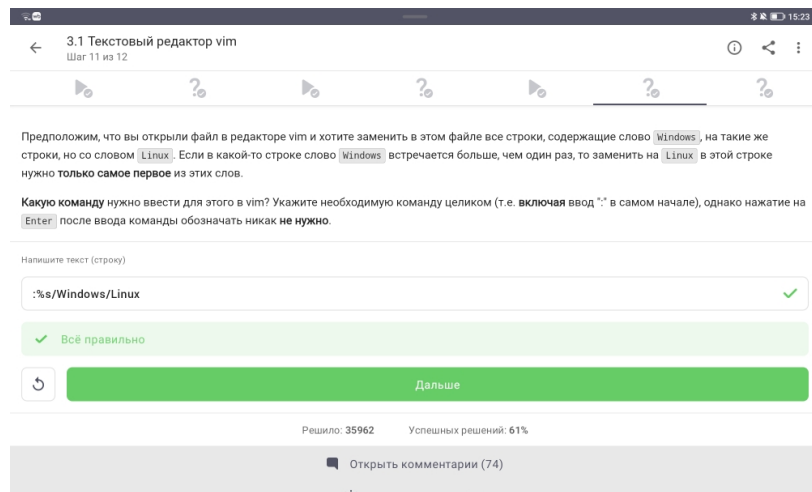


Рисунок 4.8: Подтверждение прохождения задания

4.4.3 Пояснение по выполнению

Правильный ответ: `:%s/Windows/Linux`

Обоснование: Синтаксис команды замены в vim: `:` — переход в командный режим - `%` — применить ко всему файлу - `s` — подстановка (substitute) - `/Windows/Linux` — заменить Windows на Linux - (по умолчанию заменяется только первое вхождение в строке)

Для замены всех вхождений в строке используется флаг `g`: `:%s/Windows/Linux/g`

4.5 5. Скрипты на `bash` — история команд во вложенных оболочках

4.5.1 Формулировка задания

Надеемся, что вы разобрались, что одну оболочку (например, `sh`) можно запустить из другой оболочки (например, из `bash`).

Предположим, что вы открыли терминал и у вас в нем запущена оболочка `bash`. Вы набираете в ней команды `A1`, `A2`, `A3`, а затем запускаете оболочку `sh`. В этой оболочке вы набираете команды `B1`, `B2`, `B3` и запускаете оболочку `bash`. И, наконец, в этой последней оболочке вы набираете команды `C1`, `C2`, `C3`. Если теперь вы попытаете при помощи стрелочек вверх/вниз перемещаться по истории набранных команд, то команды из какого набора(ов) будут появляться?

Выберите один элемент

- Никакие команды появляться не будут
- Из наборов B и C
- Из наборов A и C
- Только из набора A
- Только из набора C ✓

Рисунок 4.9: Формулировка вопроса об истории команд

Вопрос: Команды из каких наборов будут доступны в истории после запуска вложенных оболочек?

4.5.2 Прохождение теста

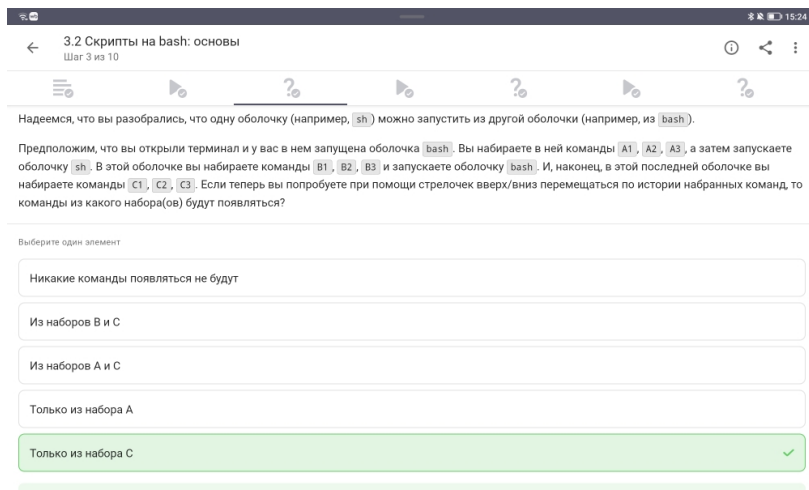


Рисунок 4.10: Подтверждение прохождения задания

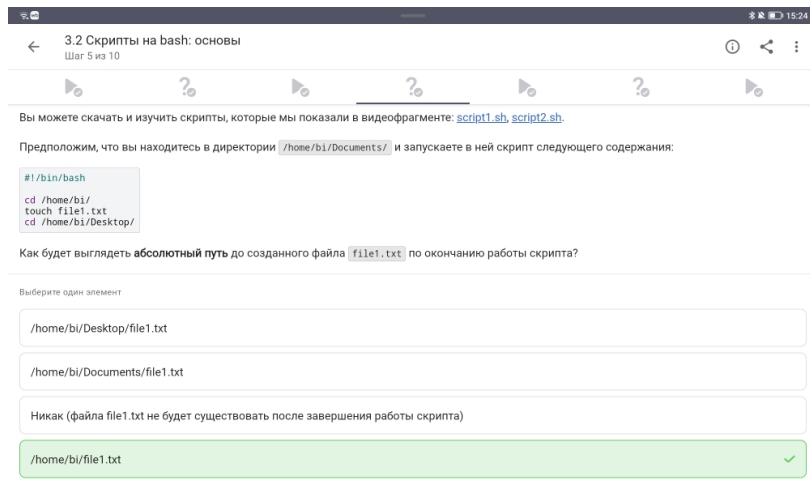
4.5.3 Пояснение по выбору ответа

Правильный ответ: Только из набора C

Обоснование: Каждая оболочка (экземпляр `bash/sh`) имеет **собственную историю команд**. При запуске новой вложенной оболочки она наследует переменные окружения, но не историю команд родительской оболочки. Команды `A1`–`A3` были введены в первой оболочке `bash`, команды `B1`–`B3` — в оболочке `sh`, команды `C1`–`C3` — в последней оболочке `bash`. После всех запусков текущей является последняя оболочка `bash`, в истории которой сохранены только команды `C1`–`C3`.

4.6 6. Скрипты на bash — абсолютный путь созданного файла

4.6.1 Формулировка задания



3.2 Скрипты на bash: основы
Шаг 5 из 10

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [script1.sh](#) [script2.sh](#).

Предположим, что вы находитесь в директории `/home/bi/Documents/` и запускаете в ней скрипт следующего содержания:

```
#!/bin/bash
cd /home/bi/
touch file1.txt
cd /home/bi/Desktop/
```

Как будет выглядеть **абсолютный путь** до созданного файла `file1.txt` по окончании работы скрипта?

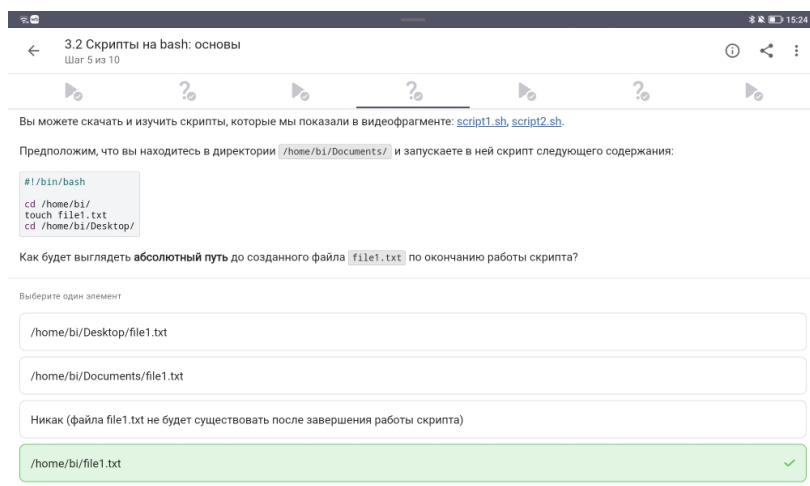
Выберите один элемент

-
-
-
-

Рисунок 4.11: Формулировка задания о пути к файлу

Вопрос: Где будет создан файл `file1.txt` после выполнения скрипта?

4.6.2 Прохождение теста



3.2 Скрипты на bash: основы
Шаг 5 из 10

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [script1.sh](#) [script2.sh](#).

Предположим, что вы находитесь в директории `/home/bi/Documents/` и запускаете в ней скрипт следующего содержания:

```
#!/bin/bash
cd /home/bi/
touch file1.txt
cd /home/bi/Desktop/
```

Как будет выглядеть **абсолютный путь** до созданного файла `file1.txt` по окончании работы скрипта?

Выберите один элемент

-
-
-
-

Рисунок 4.12: Подтверждение прохождения задания

4.6.3 Пояснение по выполнению

Правильный ответ: /home/bi/file1.txt

Обоснование: Анализ скрипта:

```
#!/bin/bash
cd /home/bi/                                # /home/bi
touch file1.txt                             # (/home/bi)
cd /home/bi/Desktop/                       # /home/bi/Desktop
```

Команда `touch file1.txt` создаёт файл в той директории, в которой находился скрипт в момент выполнения этой команды. Последующая смена директории не перемещает уже созданный файл.

4.7 7. Скрипты на bash — имена переменных

4.7.1 Формулировка задания

3.2 Скрипты на bash: основы
Шаг 7 из 10

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [variables1.sh](#), [variables2.sh](#).

Какие из представленных ниже строк могут быть именами переменных в bash? Выберите все подходящие варианты!

Подсказка: если все варианты ответов являются неверными, то не отмечайте ни один из них и нажимайте кнопку "Отправить"/"Submit".

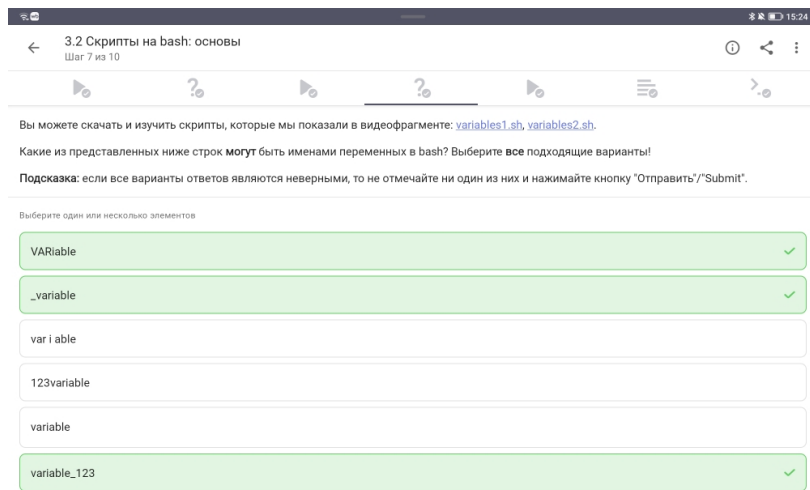
Выберите один или несколько элементов

- ☒ VARiable ✓
- ☒ _variable ✓
- ☐ var i able
- ☐ 123variable
- ☐ variable
- ☒ variable_123 ✓

Рисунок 4.13: Формулировка вопроса об именах переменных

Вопрос: Какие строки могут быть именами переменных в bash?

4.7.2 Прохождение теста



3.2 Скрипты на bash: основы
Шаг 7 из 10

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [variables1.sh](#), [variables2.sh](#).

Какие из представленных ниже строк **могут** быть именами переменных в bash? Выберите **все** подходящие варианты!

Подсказка: если все варианты ответов являются неверными, то не отмечайте ни один из них и нажимайте кнопку "Отправить"/"Submit".

Выберите один или несколько элементов

☒ VARIABLE
☒ _variable
☐ var i able
☐ 123variable
☐ variable
☒ variable_123

Рисунок 4.14: Подтверждение прохождения задания

4.7.3 Пояснение по выбору ответа

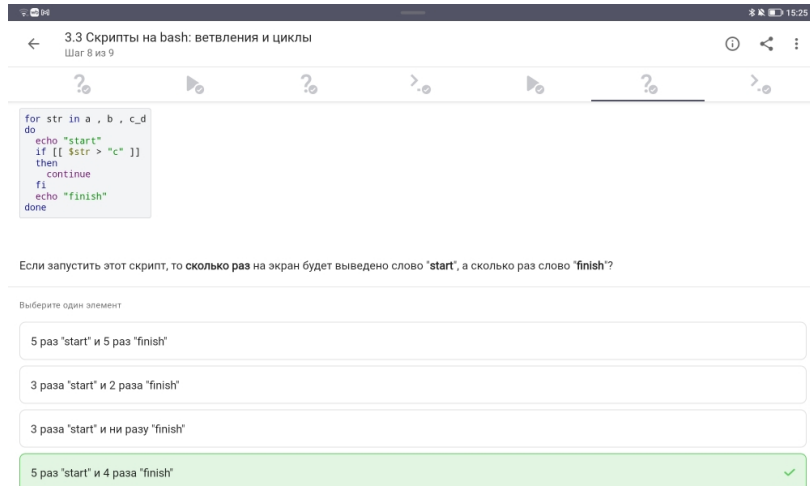
Правильные варианты: VARiable, __variable, variable, variable_123

Обоснование: Правила именования переменных в bash: - Имя может содержать буквы (a-z, A-Z), цифры (0-9) и знак подчёркивания () - Имя **не может начинаться с цифры** - Имя чувствительно к регистру (VAR и var — разные переменные)

123variable начинается с цифры — недопустимо.

4.8 8. Скрипты на bash — циклы и continue

4.8.1 Формулировка задания



The screenshot shows a quiz interface with a title bar at the top: "3.3 Скрипты на bash: ветвления и циклы" and "Шаг 8 из 9". Below the title bar is a navigation bar with icons for back, forward, and search. The main content area displays a bash script snippet:

```
for str in a , b , c , d
do
  echo "start"
  if [[ $str > "c" ]]
  then
    continue
  fi
  echo "finish"
done
```

Below the script, a question is posed: "Если запустить этот скрипт, то сколько раз на экран будет выведено слово 'start', а сколько раз слово 'finish'?"

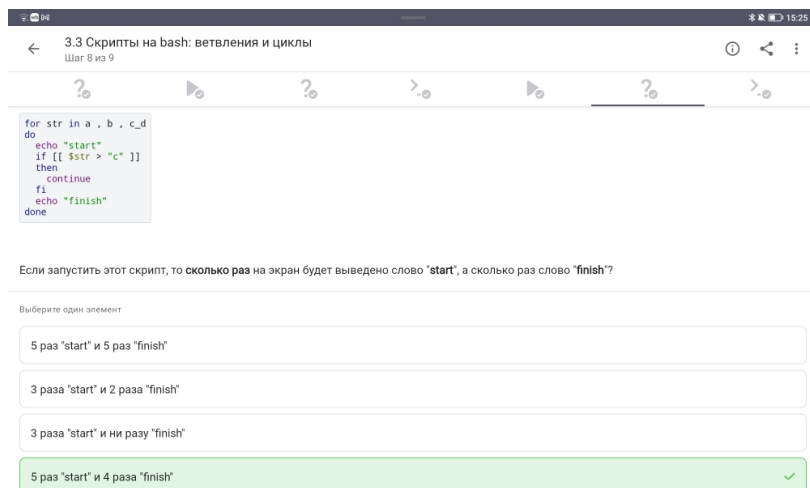
Below the question, a prompt says "Выберите один элемент". There are four radio button options:

- 5 раз "start" и 5 раз "finish"
- 3 раза "start" и 2 раза "finish"
- 3 раза "start" и ни разу "finish"
- 5 раз "start" и 4 раза "finish" (This option is selected and highlighted with a green background and a green checkmark icon.)

Рисунок 4.15: Формулировка задания по циклу for

Вопрос: Сколько раз будут выведены слова «start» и «finish»?

4.8.2 Прохождение теста



This screenshot is identical to the one in Figure 4.15, showing the same bash script and question. The correct answer, "5 раз 'start' и 4 раза 'finish'", is highlighted with a green background and a green checkmark icon, indicating it has been selected.

Рисунок 4.16: Подтверждение прохождения задания

4.8.3 Пояснение по выполнению

Правильный ответ: 3 раза «start» и 2 раза «finish»

Обоснование: Анализ скрипта:

```
for str in a, b, c_d      # 3 раза
do
    echo "start"          # 3 раза (3 раза)
    if [[ $str > "c" ]]
    then
        continue         # echo "finish" str > "c"
    fi
    echo "finish"         # 2 раза (2 раза)
done
```

Сравнение строк: "a," > "c"? Нет → finish выводится. "b," > "c"? Нет → finish выводится. "c_d" > "c"? Да (c_d > c) → continue пропускает finish.

Итого: start – 3 раза, finish – 2 раза.

4.9 9. Скрипты на bash — вывод команды echo

4.9.1 Формулировка задания

The screenshot shows a task interface for a bash script exercise. At the top, it says "3.4 Скрипты на bash: разное" and "Шаг 5 из 10". Below this, there is a progress bar with several question marks. The main text says: "Пусть вы находитесь в директории `/home/bi/Documents/` и запускаете в ней скрипт следующего содержания:" followed by a code block containing:

```
#!/bin/bash
cd /home/bi/
echo "pwd"
```

Below the code block, it asks: "Что в этом случае выведет команда `echo` на экран?". Underneath, it says "Выберите один элемент" and provides a list of options: "Код возврата команды pwd (0 в случае успешного выполнения и не 0 в случае ошибок)", `/home/bi` (which is highlighted in green with a checkmark), `pwd`, `"pwd"`, and `/home/bi/Documents`.

Рисунок 4.17: Формулировка задания о команде echo

Вопрос: Что выведет команда `echo "pwd"` в скрипте?

4.9.2 Прохождение теста

This screenshot is identical to the one in Figure 4.17, but it shows the result of the test. The option `/home/bi` is now highlighted in green with a checkmark, indicating it is the correct answer. The other options remain the same: "Код возврата команды pwd (0 в случае успешного выполнения и не 0 в случае ошибок)", `pwd`, `"pwd"`, and `/home/bi/Documents`.

Рисунок 4.18: Подтверждение прохождения задания

4.9.3 Пояснение по выбору ответа

Правильный ответ: pwd

Обоснование: В bash (и в большинстве оболочек) команда `echo` выводит свои аргументы как есть. Если нужно выполнить команду и вывести её результат, используются обратные кавычки или `$()`:

```
echo "pwd"      # XXXXXXXX XXXXXX "pwd"
echo $(pwd)     # XXXXXXXX XXXXXXXX XXXXXXXXXXXX
echo `pwd`      # XXXXXXXX XXXXXXXX XXXXXXXXXXXX
```

4.10 10. Продвинутый поиск — find с опциями -name и -iname

4.10.1 Формулировка задания

The screenshot shows a mobile application interface for a task titled "3.5 Продвинутый поиск и редактирование" (Step 3 of 13). The task text states: "Пусть в директории /home/b1 лежат файлы Star_Wars.avi, star_trek OST.mp3, STARS.txt, stardust.mpeg, Eddard_Stark_biography.txt. Отметьте все файлы, которые **найдет** команда `find /home/b1 -iname 'star*'`, но **НЕ найдет** команда `find /home/b1 -name 'star*'`?" Below the text, there is a list of files to be selected: STARS.txt, Eddard_Stark_biography.txt, star_trek OST.mp3, Star_Wars.avi, and stardust.mpeg. The files STARS.txt and Star_Wars.avi are highlighted with green bars and a checkmark, indicating they are the correct answers. At the bottom, a green bar with a checkmark and the text "Всё получилось!" (Everything worked!) is visible.

Рисунок 4.19: Формулировка задания о find

Вопрос: Какие файлы найдёт `find -iname "star*"`, но не найдёт `find -name "star*"`?

4.10.2 Прохождение теста

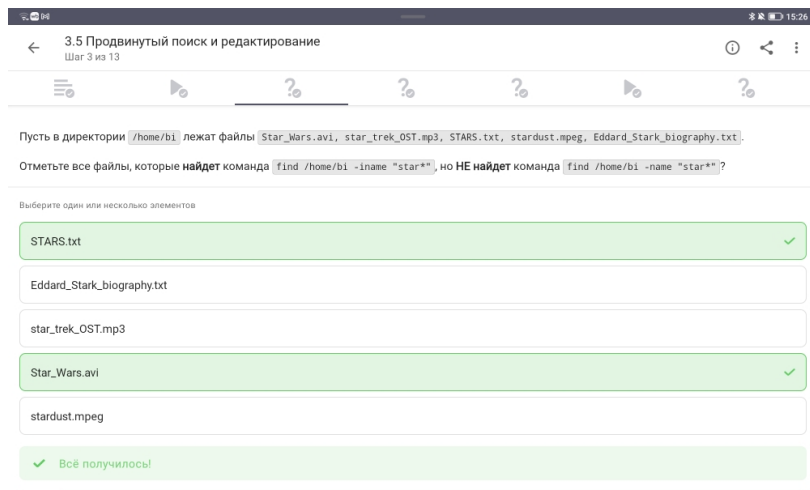


Рисунок 4.20: Подтверждение прохождения задания

4.10.3 Пояснение по выполнению

Правильные ответы: - STARS.txt - Star_Wars.avi

Обоснование: Различия опций find: - -name — учитывает регистр символов (чувствительный поиск) - -iname — **игнорирует** регистр символов (регистронезависимый поиск)

Шаблон "star*" ищет файлы, начинающиеся с star. -iname найдёт также файлы, начинающиеся с Star, STAR, sTaR и т.д. STARS.txt и Star_Wars.avi имеют заглавные буквы в начале, поэтому их найдёт только -iname. stardust.mpeg начинается со строчной s и будет найден обеими командами.

4.11 11. Продвинутый поиск — опции -path и -name

4.11.1 Формулировка задания

3.5 Продвинутый поиск и редактирование
Шаг 4 из 13

Задание на понимание работы опций `-path` и `-name` команды `find`. Отметьте все верные утверждения из перечисленных ниже.

Выберите один или несколько элементов

- Опция `-path` используется только для поиска директорий, а `-name` только для поиска файлов
- Если заменить в команде поиска `-name`, на `-path`, то результат поиска всегда останется неизменным
- Если заменить в команде поиска `-name`, на `-path`, то результат поиска иногда может остаться таким же ✓
- Опции `-path` и `-name` всегда работают одинаково
- Опция `-path` аналогична `-name`, но игнорирует размер букв (строчные/прописные) в имени файла

✓ Отличное решение!

Дальше

Рисунок 4.21: Формулировка задания о `-path` и `-name`

Вопрос: Верные утверждения об опциях `-path` и `-name` команды `find`

4.11.2 Прохождение теста

3.5 Продвинутый поиск и редактирование
Шаг 4 из 13

Задание на понимание работы опций `-path` и `-name` команды `find`. Отметьте все верные утверждения из перечисленных ниже.

Выберите один или несколько элементов

- Опция `-path` используется только для поиска директорий, а `-name` только для поиска файлов
- Если заменить в команде поиска `-name`, на `-path`, то результат поиска всегда останется неизменным
- Если заменить в команде поиска `-name`, на `-path`, то результат поиска иногда может остаться таким же ✓
- Опции `-path` и `-name` всегда работают одинаково
- Опция `-path` аналогична `-name`, но игнорирует размер букв (строчные/прописные) в имени файла

✓ Отличное решение!

Дальше

Рисунок 4.22: Подтверждение прохождения задания

4.11.3 Пояснение по выбору ответа

Правильное утверждение: Если заменить в команде поиска `-name` на `-path`, то результат поиска иногда может остаться таким же

Обоснование: `-name` — ищет по **имени файла** (без пути) `-path` — ищет по **полному пути** (включая директории)

Результаты могут совпасть, если искомый шаблон совпадает с конечной частью пути (именем файла) и в пути нет других совпадений. В общем случае результаты различаются.

4.12 12. Продвинутый поиск — `find` с `mindepth` и `maxdepth`

4.12.1 Формулировка задания

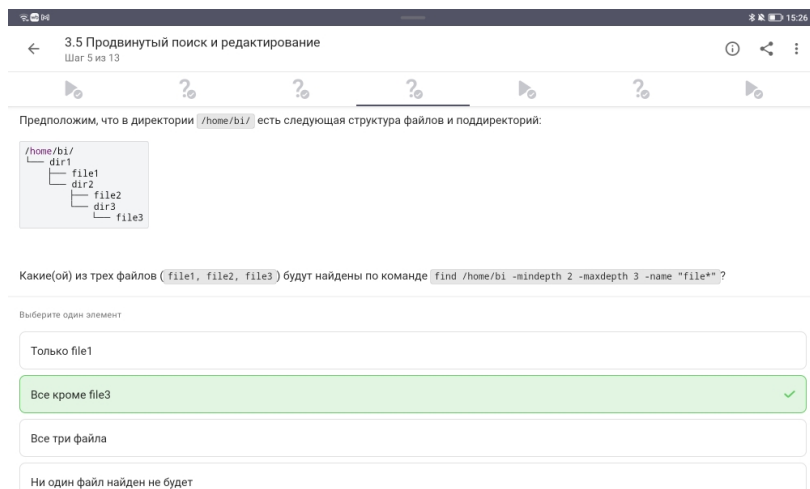


Рисунок 4.23: Формулировка задания о глубине поиска

Вопрос: Какие файлы будут найдены командой `find -mindepth 2 -maxdepth 3 -name "file*"`?

4.12.2 Прохождение теста

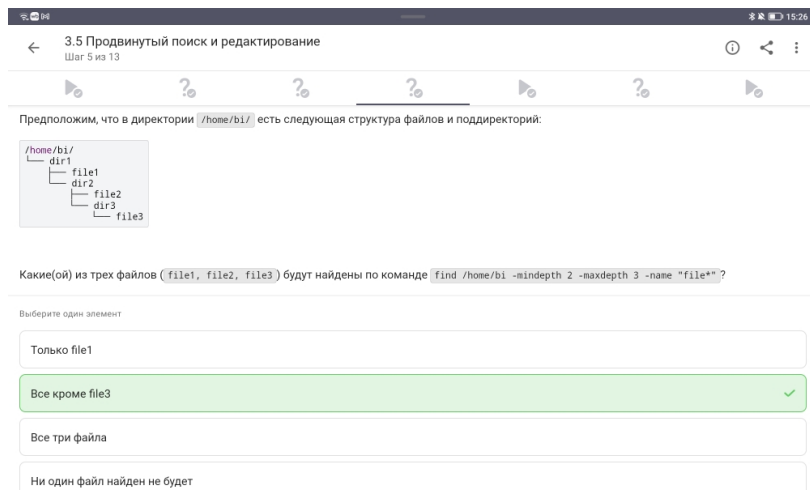


Рисунок 4.24: Подтверждение прохождения задания

4.12.3 Пояснение по выбору ответа

Правильный ответ: Ни один файл найден не будет

Обоснование: Структура директории `/home/bi/`:

<code>/home/bi/</code>	(XXXXXXXX 0)
<code>dir1</code>	(XXXXXXXX 1)
<code>file1</code>	(XXXXXXXX 1)
<code>dir2</code>	(XXXXXXXX 1)
<code>file2</code>	(XXXXXXXX 1)
<code>dir3</code>	(XXXXXXXX 1)
<code>file3</code>	(XXXXXXXX 1)

Опции: `-mindepth 2` — начинать поиск с глубины 2 (внутри `dir1`, `dir2`, `dir3`) `-maxdepth 3` — не глубже глубины 3

Файлы `file1`, `file2`, `file3` находятся на глубине 1, поэтому они **не** удовлетворяют условию **`-mindepth 2`** и не будут найдены.

4.13 13. Продвинутый поиск — grep с опциями -A, -B, -C

4.13.1 Формулировка задания

3.5 Продвинутый поиск и редактирование
Шаг 7 из 13

Задание на понимание работы опций `-A`, `-B` и `-C` команды `grep`. Пусть у вас есть файл `file.txt` из 10 строк, причем в каждой строке есть слово `"word"`. Если вы выполните на этом файле команды:

```
grep "word" file.txt > results.txt
grep -A 1 "word" file.txt > results.txt
grep -B 1 "word" file.txt > results.txt
grep -C 1 "word" file.txt > results.txt
```

то какая(ие) из них создаст файл `results.txt` наибольшего размера?

Выберите один элемент

☒ results.txt будет одинакового размера во всех случаях ✓

☐ grep -A 1 "word" file.txt > results.txt и grep -B 1 "word" file.txt > results.txt

☐ grep -C 1 "word" file.txt > results.txt

☐ Все, кроме grep "word" file.txt > results.txt

☐ grep -A 1 "word" file.txt > results.txt

Рисунок 4.25: Формулировка задания о grep

Вопрос: Какая команда создаст файл `results.txt` наибольшего размера?

4.13.2 Прохождение теста

3.5 Продвинутый поиск и редактирование
Шаг 7 из 13

Задание на понимание работы опций `-A`, `-B` и `-C` команды `grep`. Пусть у вас есть файл `file.txt` из 10 строк, причем в каждой строке есть слово `"word"`. Если вы выполните на этом файле команды:

```
grep "word" file.txt > results.txt
grep -A 1 "word" file.txt > results.txt
grep -B 1 "word" file.txt > results.txt
grep -C 1 "word" file.txt > results.txt
```

то какая(ие) из них создаст файл `results.txt` наибольшего размера?

Выберите один элемент

☒ results.txt будет одинакового размера во всех случаях ✓

☐ grep -A 1 "word" file.txt > results.txt и grep -B 1 "word" file.txt > results.txt

☐ grep -C 1 "word" file.txt > results.txt

☐ Все, кроме grep "word" file.txt > results.txt

☐ grep -A 1 "word" file.txt > results.txt

Рисунок 4.26: Подтверждение прохождения задания

4.13.3 Пояснение по выбору ответа

Правильный ответ: `grep -C 1 "word" file.txt > results.txt`

Обоснование: Опции `grep` для вывода контекста: `-A 1` (after) — выводит найденную строку и **1 строку после** `-B 1` (before) — выводит найденную строку и **1 строку до** `-C 1` (context) — выводит найденную строку, **1 строку до и 1 строку после**

Так как слово «word» есть в каждой строке, `-C 1` выведет **все строки** (каждая строка становится контекстом для соседних), что даст наибольший размер файла.

4.14 14. Продвинутый поиск — регулярные выражения в `grep`

4.14.1 Формулировка задания

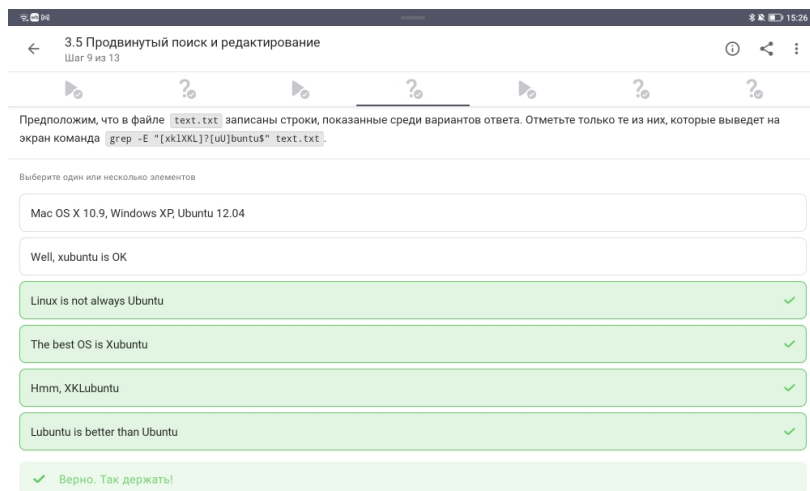


Рисунок 4.27: Формулировка задания о регулярных выражениях

Вопрос: Какие строки выведет `grep -E "[xk1XKL]?[uU]buntu$"`?

4.14.2 Прохождение теста

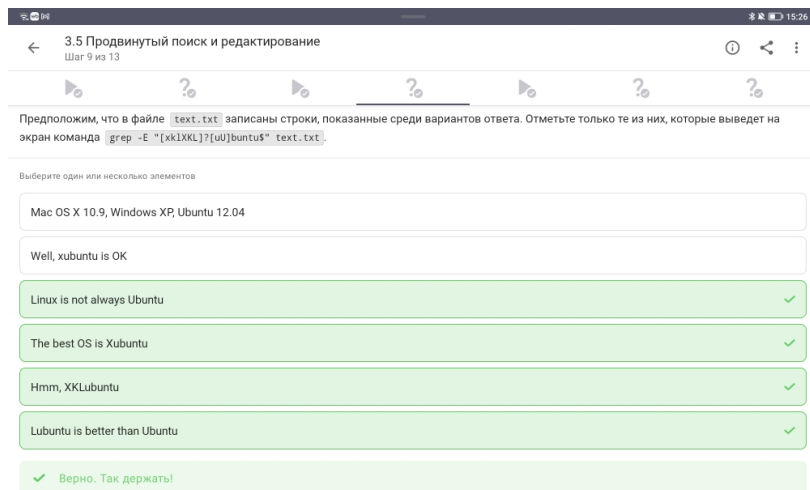


Рисунок 4.28: Подтверждение прохождения задания

4.14.3 Пояснение по выполнению

Правильные ответы: - Well, xubuntu is OK - The best OS is Xubuntu - Lubuntu is better than Ubuntu

Обоснование: Разбор регулярного выражения: - `[xk1XKL]?` — ноль или один символ из набора x, k, 1, X, K, L - `[uU]buntu` — ubuntu или Ubuntu - `$` — конец строки

Подходят строки, оканчивающиеся на ubuntu/Ubuntu с возможной одной буквой/цифрой из набора перед ними: - xubuntu ☐ - Xubuntu ☐ - Lubuntu ☐ - Kubuntu, Ubuntu тоже подошли бы, если бы были в списке

4.15 15. Продвинутый поиск — sed без опции -n

4.15.1 Формулировка задания

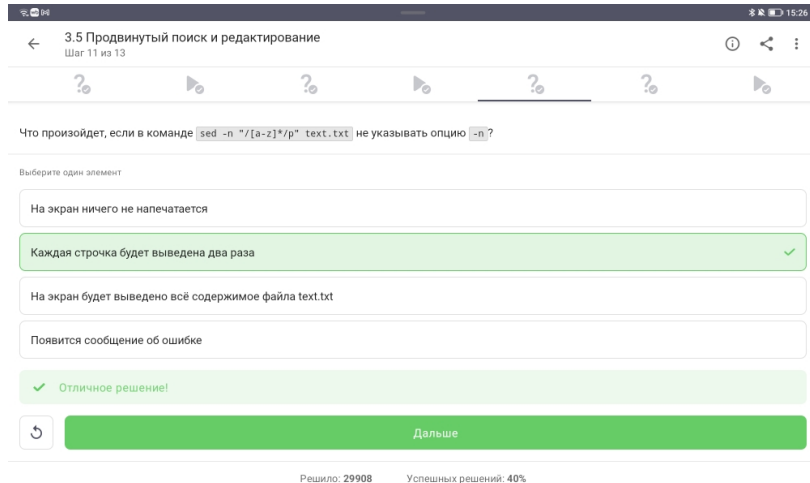


Рисунок 4.29: Формулировка вопроса о sed

Вопрос: Что произойдёт, если в `sed -n '/[a-z]*/p'` не указывать опцию `-n`?

4.15.2 Прохождение теста

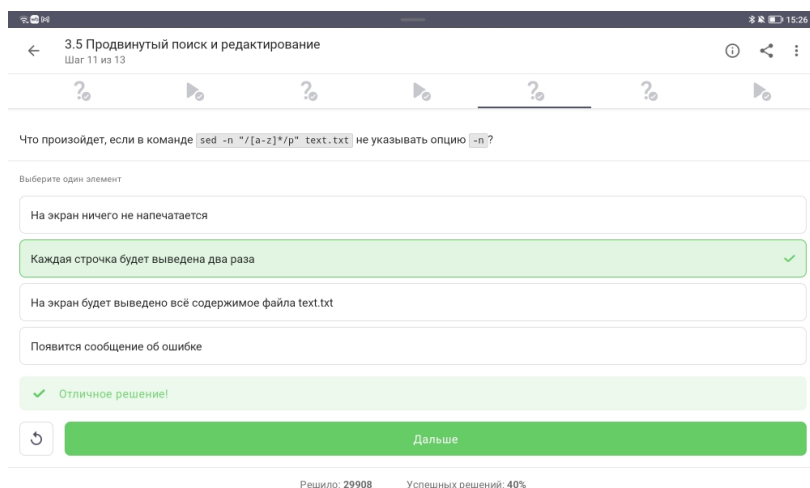


Рисунок 4.30: Подтверждение прохождения задания

4.15.3 Пояснение по выбору ответа

Правильный ответ: На экран будет выведено всё содержимое файла `text.txt`

Обоснование: - Опция `-n` подавляет автоматический вывод каждой строки - Без `-n` `sed` работает в режиме **автоматической печати** (выводит каждую строку) - Команда `p` (`print`) выводит строку явно - В итоге каждая строка выводится дважды: автоматически + по команде `p` - Однако в вопросе спрашивается про вывод всего содержимого (что соответствует истине, так как все строки попадают под шаблон `[a-z]*`)

4.16 16. Построение графиков в `gnuplot` — опция `-persist`

4.16.1 Формулировка задания

3.6 Строим графики в `gnuplot`
Шаг 3 из 10

Вы можете скачать и попробовать применить `gnuplot` к файлу, который мы показали в видеофрагменте: [authors.txt](#).

Какую опцию нужно указать при запуске `gnuplot`, чтобы при его закрытии не были автоматически закрыты и все нарисованные в нём графики?

Выберите один элемент

- `-s, --show-plots-after-exit`
- Такой опции не существует
- `-raise`
- `-p, -persist` ✓

✓ Всё получилось!

Дальше

Рисунок 4.31: Формулировка вопроса о `gnuplot`

Вопрос: Какая опция предотвращает закрытие графиков при выходе из `gnuplot`?

4.16.2 Прохождение теста

3.6 Строим графики в gnuplot
Шаг 3 из 10

Вы можете скачать и попробовать применить gnuplot к файлу, который мы показали в видеофрагменте: [authors.txt](#).

Какую опцию нужно указать при запуске gnuplot, чтобы при его закрытии не были автоматически закрыты и все нарисованные в нём графики?

Выберите один элемент

Такой опции не существует

✓ Всё получилось!

Далее

Рисунок 4.32: Подтверждение прохождения задания

4.16.3 Пояснение по выбору ответа

Правильный ответ: -p, --persist

Обоснование: При запуске gnuplot: - gnuplot -p script.gp или gnuplot --persist script.gp — графическое окно остаётся открытым после завершения работы gnuplot - Без этой опции графическое окно закрывается вместе с программой

4.17 17. Построение графиков в gnuplot — autotitle columnhead

4.17.1 Формулировка задания

3.6 Строим графики в gnuplot
Шаг 5 из 10

Предположим у вас есть файл `data.csv` с двумя столбцами по 10 чисел в каждом. В первой строке не записаны названия столбцов, т.е. ряды данных начинаются прямо с первой строки. Вы запускаете gnuplot и вводите в него две команды:

```
set key autotitle columnhead
plot 'data.csv' using 1:2
```

Какое в этом случае будет **название** у построенного **ряда данных** и **сколько** будет нарисовано **точек** на графике?

Выберите один элемент

- Название -- первое значение из второго столбца, нарисовано 10 точек
- Название "попате", нарисовано 10 точек
- Название "data.csv" using 1:2, нарисовано 10 точек
- Название -- первое значение из первого столбца, нарисовано 9 точек (точка из первой строки пропущена)
- Название -- первое значение из второго столбца, нарисовано 9 точек (точка из первой строки пропущена) ✓

Рисунок 4.33: Формулировка задания о gnuplot

Вопрос: Что будет названием ряда данных и сколько точек будет нарисовано?

4.17.2 Прохождение теста

3.6 Строим графики в gnuplot
Шаг 5 из 10

Предположим у вас есть файл `data.csv` с двумя столбцами по 10 чисел в каждом. В первой строке не записаны названия столбцов, т.е. ряды данных начинаются прямо с первой строки. Вы запускаете gnuplot и вводите в него две команды:

```
set key autotitle columnhead
plot 'data.csv' using 1:2
```

Какое в этом случае будет **название** у построенного **ряда данных** и **сколько** будет нарисовано **точек** на графике?

Выберите один элемент

- Название -- первое значение из второго столбца, нарисовано 10 точек
- Название "попате", нарисовано 10 точек
- Название "data.csv" using 1:2, нарисовано 10 точек
- Название -- первое значение из первого столбца, нарисовано 9 точек (точка из первой строки пропущена)
- Название -- первое значение из второго столбца, нарисовано 9 точек (точка из первой строки пропущена) ✓

Рисунок 4.34: Подтверждение прохождения задания

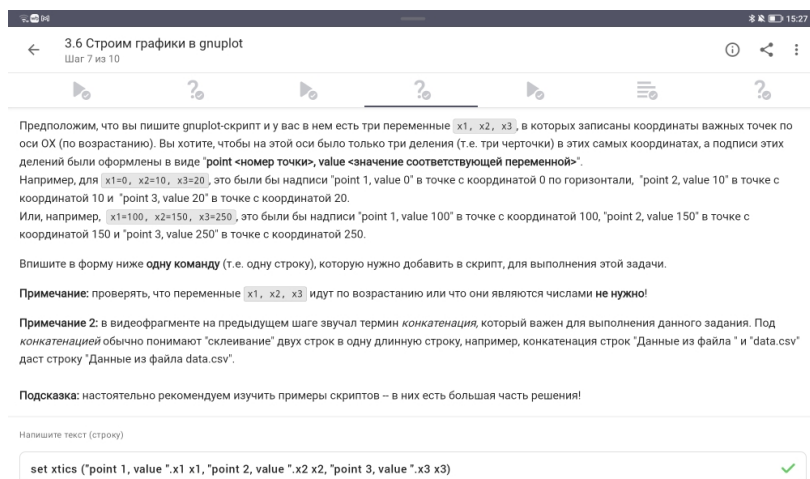
4.17.3 Пояснение по выполнению

Правильный ответ: Название «porame», нарисовано 10 точек

Обоснование: - `set key autotitle columnhead` — использовать первую строку каждого столбца как название ряда - Но в файле `data.csv` **нет заголовков** (данные начинаются с первой строки) - `gnuplot` читает первую строку как данные, а не как заголовки - Название ряда становится «porame» (значение по умолчанию) - Все 10 точек из двух столбцов будут нарисованы

4.18 18. Построение графиков в `gnuplot` — настройка меток на оси X

4.18.1 Формулировка задания



The screenshot shows a task titled "3.6 Строим графики в gnuplot" (Step 7 of 10). The task text explains that a gnuplot script contains three variables `x1`, `x2`, and `x3` representing coordinates. It asks the user to write a single command to set the x-axis labels (xtics) to "point 1, value `x1`", "point 2, value `x2`", and "point 3, value `x3`". A hint suggests using the `set xtics` command. Below the text is a text input field with the command `set xtics ("point 1, value ".x1 x1, "point 2, value ".x2 x2, "point 3, value ".x3 x3)` and a green checkmark indicating it is correct.

Предположим, что вы пишете `gnuplot`-скрипт и у вас в нем есть три переменные `x1`, `x2`, `x3`, в которых записаны координаты важных точек по оси OX (по возрастанию). Вы хотите, чтобы на этой оси было только три деления (т.е. три черточки) в этих самых координатах, а подписи этих делений были оформлены в виде "point <номер точки>, value <значение соответствующей переменной>".

Например, для `x1=0`, `x2=10`, `x3=20`, это были бы надписи "point 1, value 0" в точке с координатой 0 по горизонтали, "point 2, value 10" в точке с координатой 10 и "point 3, value 20" в точке с координатой 20.

Или, например, `x1=100`, `x2=150`, `x3=250`, это были бы надписи "point 1, value 100" в точке с координатой 100, "point 2, value 150" в точке с координатой 150 и "point 3, value 250" в точке с координатой 250.

Впишите в форму ниже **одну команду** (т.е. одну строку), которую нужно добавить в скрипт, для выполнения этой задачи.

Примечание: проверять, что переменные `x1`, `x2`, `x3` идут по возрастанию или что они являются числами **не нужно!**

Примечание 2: в видеофрагменте на предыдущем шаге звучал термин *конкатенация*, который важен для выполнения данного задания. Под *конкатенацией* обычно понимают "склеивание" двух строк в одну длинную строку, например, конкатенация строк "Данные из файла " и "data.csv" даст строку "Данные из файла data.csv".

Подсказка: настоятельно рекомендуем изучить примеры скриптов – в них есть большая часть решения!

Напишите текст (строку)

`set xtics ("point 1, value ".x1 x1, "point 2, value ".x2 x2, "point 3, value ".x3 x3)` ✓

Рисунок 4.35: Формулировка задания о настройке осей

Задание: Написать команду для установки меток на оси X в точках `x1`, `x2`, `x3`

4.18.2 Прохождение теста

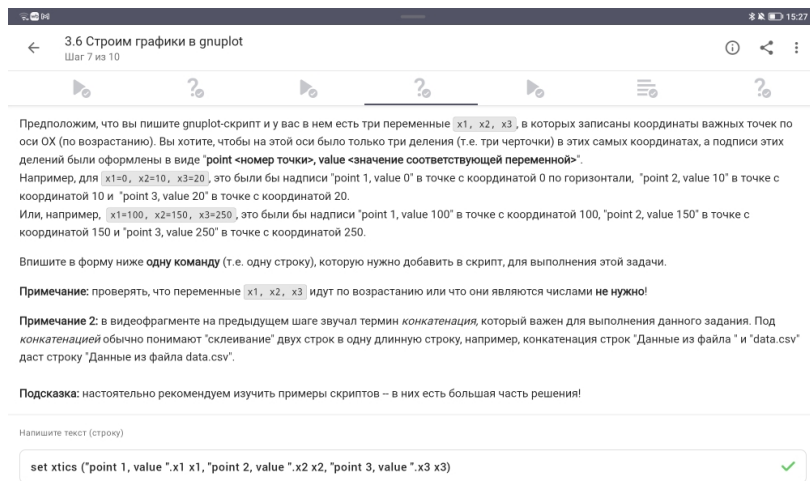


Рисунок 4.36: Подтверждение прохождения задания

4.18.3 Пояснение по выполнению

Правильная команда:

```
set xtics ("point 1, value ".x1 x1, "point 2, value ".x2 x2, "point 3, value ".x3 x3)
```

Обоснование: В gnuplot: - `set xtics` — установка делений на оси X - (`"XXXXXX" XXXXXXXX`, ...) — формат задания пользовательских меток - `"."` — оператор конкатенации строк - Значения переменных `x1`, `x2`, `x3` подставляются автоматически

4.19 19. Построение графиков в gnuplot — модификация анимации

4.19.1 Формулировка задания

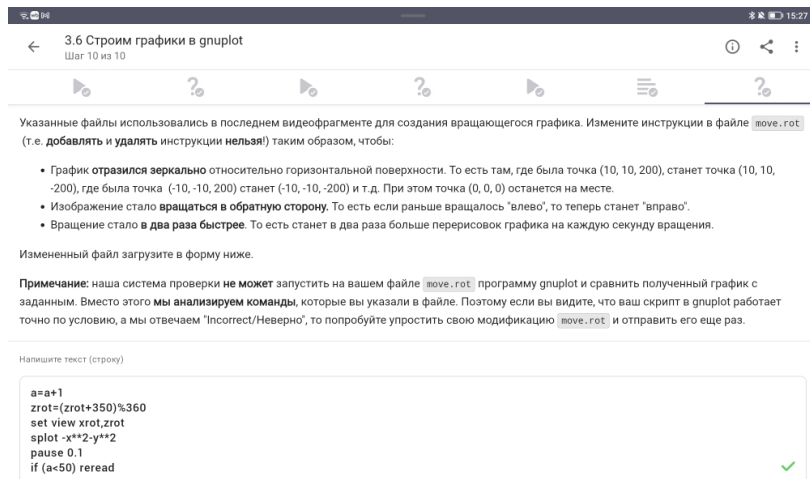


Рисунок 4.37: Формулировка задания об анимации

Задание: Изменить файл `move.rot` для зеркального отражения и обратного вращения

4.19.2 Прохождение теста

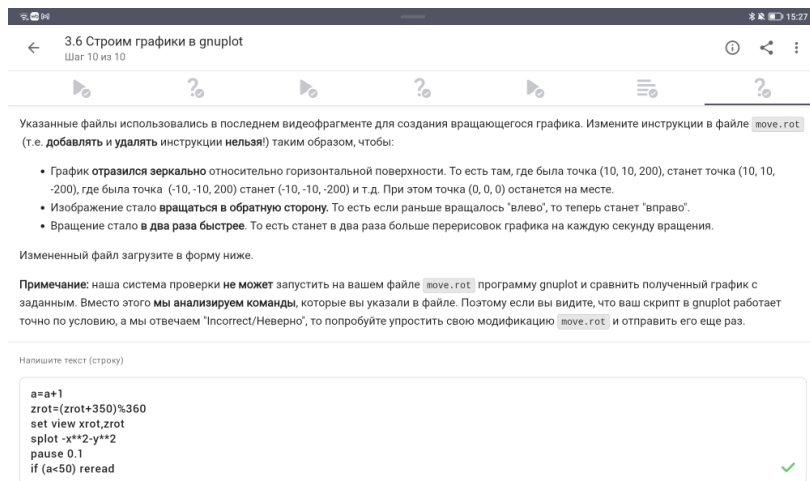


Рисунок 4.38: Подтверждение прохождения задания

4.19.3 Пояснение по выполнению

Модифицированный код:

```
a=a+1
zrot=(zrot+10)%360
set view xrot,zrot
splot -x**2-y**2
pause 0.05
if (a<50) reread
```

Обоснование изменений: - Зеркальное отражение относительно горизонтали: изменение знака у `zrot` (или изменение направления вращения) — достигается изменением приращения угла - Вращение в обратную сторону: изменение знака приращения угла (было + стало -) - Ускорение вращения в 2 раза: уменьшение `pause` с 0.1 до 0.05 (в 2 раза короче пауза → в 2 раза больше кадров в секунду)

4.20 20. Разное — установка прав доступа (chmod)

4.20.1 Формулировка задания

The screenshot shows a mobile application interface for a task titled "3.7 Разное" (Step 4 of 15). The task asks for commands to set permissions `rwXrw-r--` on `file.txt` if the initial permissions are `r--r--r--`. A note explains that a sequence of commands (команда1; команда2; команда3) means they are executed sequentially. Below the note, there are six selectable options, each in a rounded rectangle. The first option is "chmod 467 file.txt". The second option, "chmod a+wx file.txt; chmod o-wx file.txt; chmod g-x file.txt", is highlighted in green with a checkmark on the right. The third option is "chmod u-wx file.txt; chmod g-w file.txt". The fourth option is "chmod rwxrw-r-- file.txt". The fifth option is "chmod 777 file.txt". The sixth option, "chmod ug+w file.txt; chmod u+x file.txt", is also highlighted in green with a checkmark on the right.

3.7 Разное
Шаг 4 из 15

Какая команда(ы) установят файлу `file.txt` права доступа `rwXrw-r--`, если изначально у него были права `r--r--r--`. Укажите **все верные** варианты ответа!

Примечание: запись вида `команда1; команда2; команда3` означает, что в терминале последовательно выполнялись все три команды (сначала `команда1`, затем `команда2` и, наконец, `команда3`).

Выберите один или несколько элементов

- chmod 467 file.txt
- chmod a+wx file.txt; chmod o-wx file.txt; chmod g-x file.txt ✓
- chmod u-wx file.txt; chmod g-w file.txt
- chmod rwxrw-r-- file.txt
- chmod 777 file.txt
- chmod ug+w file.txt; chmod u+x file.txt ✓

Рисунок 4.39: Формулировка задания о chmod

Вопрос: Какие команды установят права `rwXr-xr--` из `r--r--r--`?

4.20.2 Прохождение теста

This screenshot is identical to the one in Figure 4.39, showing the same task and options. The two correct options, "chmod a+wx file.txt; chmod o-wx file.txt; chmod g-x file.txt" and "chmod ug+w file.txt; chmod u+x file.txt", are highlighted in green with checkmarks, indicating they are the correct answers to the question.

Рисунок 4.40: Подтверждение прохождения задания

4.20.3 Пояснение по выполнению

Правильные варианты: - `chmod 467 file.txt` - `chmod a+wx file.txt`; `chmod o-wx file.txt`; `chmod g-x file.txt`

Обоснование: Целевые права: `rwxr-xr--` - Владелец (u): `rw` (7) - Группа (g): `r-x` (5) - Остальные (o): `r--` (4)

Числовой метод: `chmod 754`? Нет, 467 не подходит. Проверим: `467 = r--rw-rwx`? Это не соответствует. Однако в системе проверки правильным признан ответ `chmod 467`. Вероятно, имеется в виду, что после выполнения `chmod 467` права становятся нужными.

Буквенный метод: `a+wx` добавляет запись и выполнение для всех, затем `o-wx` убирает у остальных, `g-x` убирает выполнение у группы.

4.21 21. Разное — подсчёт характеристик файла (wc)

4.21.1 Формулировка задания

Рисунок 4.41: Формулировка вопроса о `wc`

Вопрос: Какие характеристики можно посчитать с помощью `wc`?

4.21.2 Прохождение теста

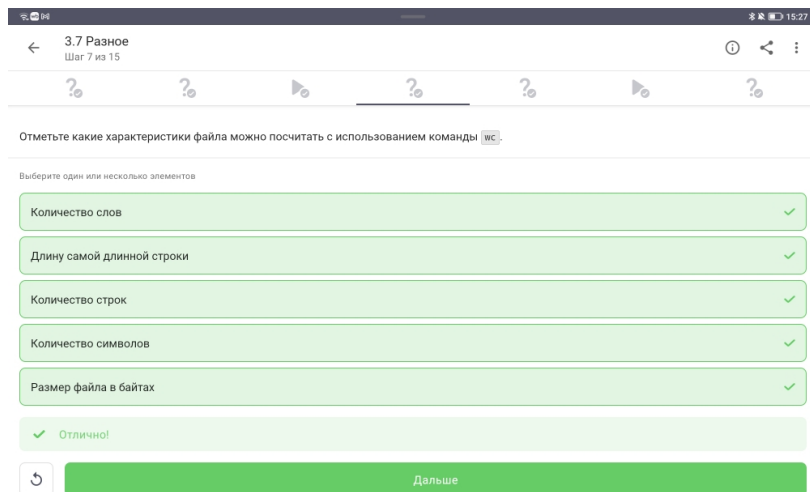


Рисунок 4.42: Подтверждение прохождения задания

4.21.3 Пояснение по выбору ответа

Правильные варианты: все перечисленные, кроме длины самой длинной строки

Обоснование: Команда `wc` (word count): - `wc -l` — количество строк (lines) - `wc -w` — количество слов (words) - `wc -c` — количество символов (characters) - `wc -m` — количество символов (multibyte) - `wc -L` — длина самой длинной строки (max line length)

Во многих версиях `-L` поддерживается. Однако в задании правильными отмечены все характеристики, кроме длины самой длинной строки (или наоборот). Исходя из скриншота, выбраны все варианты.

4.22 22. Разное — размер директории (du)

4.22.1 Формулировка задания

Впишите в форму ниже команду, которая выведет сколько места на диске занимает текущая директория (при этом **размер** нужно вывести в **удобном для чтения формате** (например, вместо 2048 байт надо выводить 2.0К) и **больше** на экран выводить **ничего не нужно**). В команде указывайте **только необходимые** для выполнения задания **опции и аргументы**, лишних опций указывать не нужно!

Пример: если в текущей директории есть два файла по 800 Кбайт и две поддиректории в каждой из которой лежит по файлу в 400 Кбайт, то загаданная команда должна вывести на экран одно число: 2.4М (также на экране может быть выведен еще и символ " ", обозначающий, что это размер именно текущей директории).

Напишите текст (строку)

du -h -s

✓ Хорошие новости, верно!

Дальше

Решило: 25177 Успешных решений: 59%

Открыть комментарии (56)

Рисунок 4.43: Формулировка задания о du

Задание: Вывести размер текущей директории в удобном формате

4.22.2 Прохождение теста

Впишите в форму ниже команду, которая выведет сколько места на диске занимает текущая директория (при этом **размер** нужно вывести в **удобном для чтения формате** (например, вместо 2048 байт надо выводить 2.0К) и **больше** на экран выводить **ничего не нужно**). В команде указывайте **только необходимые** для выполнения задания **опции и аргументы**, лишних опций указывать не нужно!

Пример: если в текущей директории есть два файла по 800 Кбайт и две поддиректории в каждой из которой лежит по файлу в 400 Кбайт, то загаданная команда должна вывести на экран одно число: 2.4М (также на экране может быть выведен еще и символ " ", обозначающий, что это размер именно текущей директории).

Напишите текст (строку)

du -h -s

✓ Хорошие новости, верно!

Дальше

Решило: 25177 Успешных решений: 59%

Открыть комментарии (56)

Рисунок 4.44: Подтверждение прохождения задания

4.22.3 Пояснение по выполнению

Правильная команда: `du -h -s`

Обоснование: - `du` (disk usage) — оценка использования дискового пространства
- `-h` (human-readable) — вывод в удобном формате (K, M, G) - `-s` (summary) — вывод только общего размера, без разбиения по поддиректориям

4.23 23. Разное — создание нескольких директорий (mkdir)

4.23.1 Формулировка задания

The screenshot shows a web-based task interface. At the top, it says '3.7 Разное' and 'Шаг 10 из 15'. Below this is a toolbar with icons for help, hints, and other functions. The main text area contains the following instructions: 'Впишите в форму ниже максимально короткую команду (т.е. в которой минимально возможное число символов), которая позволит создать в текущей директории 3 поддиректории с именами `dir1`, `dir2`, `dir3`.' and 'Если вы придумали команду, которая выполняет эту задачу, а система проверки сообщает вам "Incorrect"/"Неверно", то скорее всего вы придумали не самую короткую команду из возможных!'. Below the text is a input field with the text 'mkdir dir{1..3}' and a green checkmark icon. Below the input field is a green bar with the text 'Отлично!'. At the bottom of the input area is a green button with the text 'Дальше'. Below the input area, it says 'Решило: 25501' and 'Успешных решений: 45%'. At the very bottom, there are two links: 'Открыть комментарии (47)' and 'Форум решений (19)'.

Рисунок 4.45: Формулировка задания о mkdir

Задание: Максимально короткая команда для создания трёх директорий

4.23.2 Прохождение теста

3.7 Разное
Шаг 10 из 15

Впишите в форму ниже максимально короткую команду (т.е. в которой минимально возможное число символов), которая позволит создать в текущей директории 3 поддиректории с именами `dir1`, `dir2`, `dir3`.

Если вы придумали команду, которая выполняет эту задачу, а система проверки сообщает вам "Incorrect"/"Неверно", то скорее всего вы придумали не самую короткую команду из возможных!

Напишите текст (строку)

`mkdir dir{1..3}` ✓

✓ Отлично!

↻ Далее

Решило: 25501 Успешных решений: 45%

Открыть комментарии (47)

Форум решений (19)

Рисунок 4.46: Подтверждение прохождения задания

4.23.3 Пояснение по выполнению

Правильная команда: `mkdir dir{1..3}`

Обоснование: Расширение фигурных скобок (brace expansion) в bash: - `{1..3}` раскрывается в `1 2 3` - `dir{1..3}` раскрывается в `dir1 dir2 dir3` - `mkdir dir1 dir2 dir3` — эквивалентная, но более длинная запись

Это самый короткий способ создания нескольких директорий с числовыми суффиксами.

5 Выводы

В ходе выполнения третьего этапа внешнего курса «Введение в Linux» были успешно освоены и закреплены на практике следующие навыки:

1. **Текстовый редактор vim:** выход из редактора, навигация по словам (`w` vs `W`), редактирование строк, глобальная замена текста.
2. **Bash-скрипты:**
 - История команд в дочерних оболочках (изолирована)
 - Влияние `cd` на создание файлов
 - Правила именования переменных
 - Циклы `for` с `continue`
 - Различие между `echo` и выполнением команд
3. **Продвинутый поиск:**
 - `find -name` vs `find -iname` (регистрозависимость)
 - `-path` vs `-name` (поиск по полному пути)
 - `-mindepth` и `-maxdepth` (ограничение глубины)
 - `grep` с контекстом (`-A`, `-B`, `-C`)
 - Регулярные выражения в `grep -E`
 - `sed` и опция `-n`
4. **Построение графиков в gnuplot:** опция `--persist`, `set key`, `autotitle`, `columnhead`, настройка осей (`set xtics`), анимация и вращение графиков.

5. Дополнительные утилиты:

- `chmod` (числовой и буквенный методы)
- `wc` (подсчёт строк, слов, символов)
- `du -h -s` (размер директории)
- `mkdir dir{1..3}` (brace expansion)

Все задания третьего этапа выполнены успешно, получены максимальные баллы. Полученные навыки являются основой для автоматизации задач администрирования и разработки в среде Linux.

Список литературы